Generative AI Part-2

Deepika Kanchi

October 2026

Contents

1 Generative AI Models **2**
1.1 GPT – Generative Pre-trained Transformer 2
1.1.1 How GPT Works 2
1.1.2 GPT Architecture 2
1.1.3 Autoregressive Property 3
1.1.4 Self-Attention 3
1.1.5 Applications 3
1.2 Codex 4
1.2.1 Key Features 4
1.2.2 Architecture: 4
1.2.3 Mathematical Formulation 4
1.2.4 Supported Programming Languages 5
1.2.5 Applications: 5
1.2.6 GitHub Copilot and Codex 5
1.3 DALL.E 6
1.3.1 Key Features 6
1.3.2 Architecture of DALL-E 6
1.3.3 Working of DALL-E 6
1.4 Stable Diffusion: 7
1.4.1 Architecture of Stable Diffusion 7
1.4.2 Working of Stable Diffusion 8

1 Generative AI Models

- GPT generates text using Transformers.
- Codex is GPT trained specifically on code.
- DALL·E generates images from text prompts.
- Stable Diffusion generates images using a diffusion process that gradually removes noise to create realistic images.

1.1 GPT – Generative Pre-trained Transformer

GPT (Generative Pre-trained Transformer) is a decoder-only Transformer-based Large Language Model that generates text by predicting the next token from previous tokens. It is first pre-trained on large text corpora and then fine-tuned to perform tasks such as question answering, summarization, translation, and code generation.

1.1.1 How GPT Works

Pre-training: The model learns language patterns, grammar, and knowledge from a large collection of text data. Its primary objective is to predict the next token.

Example:

I love drinking _____

Possible predictions:

- Coffee
- Tea
- Water

Fine-tuning: After pre-training, the model is further trained using task-specific data and human feedback to improve accuracy, safety, and usefulness.

1.1.2 GPT Architecture

GPT is a **Decoder-Only Transformer**.

Components

1. **Tokenization**
   - Input text is broken into tokens.
2. **Embedding Layer**

- Converts tokens into dense numerical vectors.

3. **Positional Encoding**
   - Provides information about word order.
4. **Masked Multi-Head Self-Attention**
   - Allows the model to attend only to previous tokens.
   - Captures relationships between words.
5. **Feed Forward Network (FFN)**
   - Further processes the information.
6. **Layer Normalization and Residual Connections**
   - Help stabilize training.
7. **Softmax Layer**
   - Produces probabilities for the next token prediction.

1.1.3 Autoregressive Property

GPT predicts text one token at a time based on previously generated tokens.

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t | x_1, \ldots, x_{t-1}) \tag{1}$$

This means that each token is predicted using all the previous tokens.

1.1.4 Self-Attention

Self-attention helps GPT identify relationships between words in a sentence. **Example:**

> The animal didn't cross the street because it was tired.

The model understands that the word "it" refers to "animal".

1.1.5 Applications

- ChatGPT
- Microsoft Copilot
- GitHub Copilot
- Content Generation
- Customer Service Chatbots

- Language Translation
- Text Summarization
- Code Generation

1.2 Codex

Codex is a Generative AI model developed by OpenAI that is specifically designed for programming tasks. It is built upon the GPT architecture and trained on large amounts of source code and natural language text. Codex can understand programming requirements written in natural language and generate executable code.

Codex = GPT specialized for coding tasks.

1.2.1 Key Features

- Generates code from natural language instructions.
- Supports multiple programming languages.
- Performs code completion.
- Generates unit tests.
- Assists in debugging.
- Explains existing code.
- Converts code between programming languages.

1.2.2 Architecture:

Codex uses a Decoder-Only Transformer Architecture as GPT

1.2.3 Mathematical Formulation

Codex models code generation as a sequence prediction problem:

$$P(c_1, c_2, \ldots, c_n) = \prod_{t=1}^{n} P(c_t | c_1, c_2, \ldots, c_{t-1}) \tag{2}$$

where:

- c_t represents the current code token.
- c_{t-1} represents previously generated code tokens.

This means that Codex predicts the next code token using all previously generated tokens.

1.2.4 Supported Programming Languages

Codex supports various programming languages including:

- Python
- Java
- JavaScript
- C
- C++
- C#
- Go
- PHP
- Ruby
- SQL
- TypeScript

1.2.5 Applications:

- Code Generation: Generates code directly from natural language instructions.
- Code Completion: Suggests code snippets while developers write programs.
- Debugging: Identifies coding errors and suggests fixes.
- Unit Test Generation: Automatically produces test cases for software applications.
- Code Translation: Converts code from one programming language to another.

1.2.6 GitHub Copilot and Codex

One of the most popular applications of Codex is GitHub Copilot. It assists developers by generating code suggestions, completing functions, and creating boilerplate code directly within code editors.

1.3 DALL.E

DALL-E is a Generative Artificial Intelligence model developed by OpenAI that generates images from textual descriptions. It is a text-to-image model capable of creating realistic and creative images based on natural language prompts.

Example: Input:

"A Bear Playing a Piano"

Output:

An AI-generated image of a bear playing a piano.

1.3.1 Key Features

- Generates images from text prompts.
- Produces realistic and creative artwork.
- Understands objects, styles, colors, and scenes.
- Supports image editing and variations.
- Creates multiple image outputs from a single prompt.

1.3.2 Architecture of DALL-E

DALL-E is based on the Transformer architecture and consists of the following components:

1. Text Encoder
2. CLIP Model
3. Prior Network
4. Decoder

1.3.3 Working of DALL-E

Step 1: Text Prompt The user provides a textual description.

"A Bear Playing a Piano"

Step 2: Text Encoding The text encoder converts the prompt into a numerical representation.

Step 3: CLIP Understanding CLIP learns the relationship between text and images. It aligns image embeddings and text embeddings in a common representation space

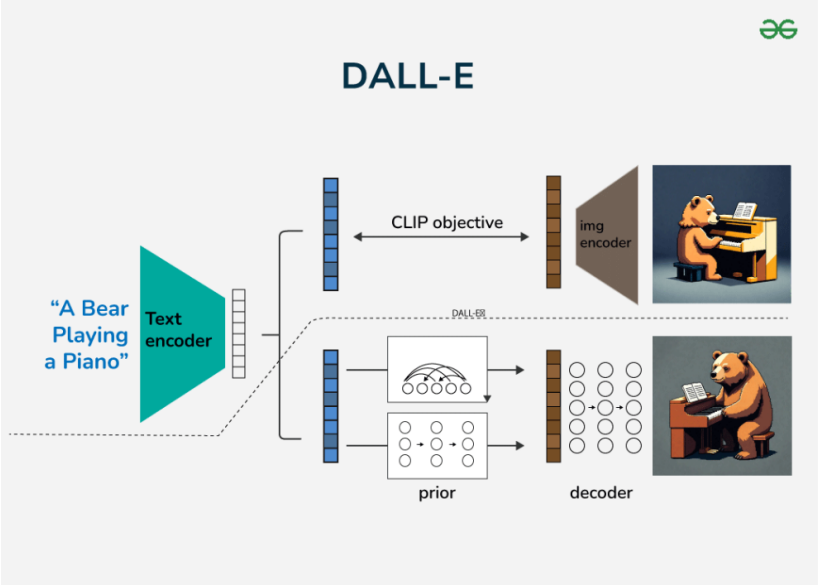


Figure 1: DALL.e image

Step 4: Prior Network The Prior network converts the text embedding into an image embedding. this stage allows DALL-E to imagine what the final image should contain.

Step 5: Decoder The decoder converts the image embedding into actual pixels.

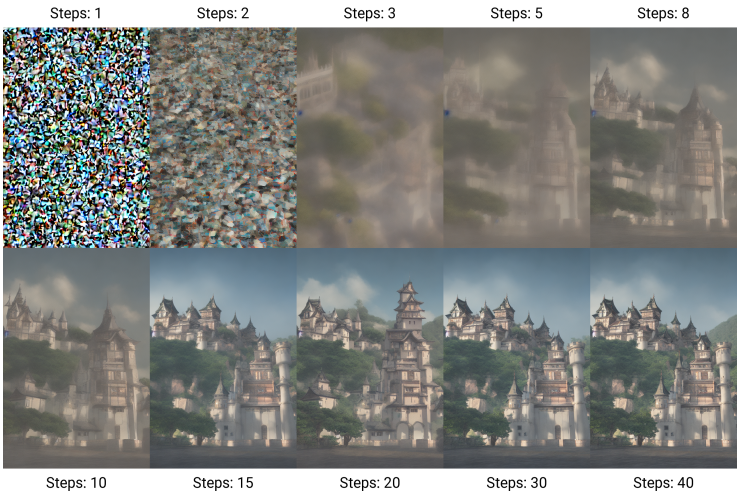
1.4 Stable Diffusion:

Stable Diffusion is a Generative Artificial Intelligence model used for text-to-image generation. It was released in 2022 by Stability AI and is based on Latent Diffusion Models (LDMs). It generates realistic images from textual descriptions by gradually removing noise from a latent representation.

1.4.1 Architecture of Stable Diffusion

The main components are:

1. CLIP Text Encoder
2. Latent Space
3. U-Net
4. Variational Autoencoder (VAE)



1.4.2 Working of Stable Diffusion

Step 1: Text Prompt The user enters a text prompt.

"A bear playing a piano"

Step 2: Text Encoding The CLIP text encoder converts the prompt into a text embedding.

Step 3: Generate Random Latent Noise The model starts with random noise in latent space.

$$z_T \sim \mathcal{N}(0, I)$$

where

- z_T = noisy latent representation
- $\mathcal{N}(0, I)$ = Gaussian noise

Step 4: U-Net Denoising The U-Net progressively removes noise using the text embedding as guidance.

$$z_t \rightarrow z_{t-1}$$

The denoising process gradually transforms noise into a meaningful latent image.

Step 5: Final Latent Image After multiple denoising steps, a clean latent image is produced.

Step 6: VAE Decoder The VAE decoder converts the latent image into a real image.